

Features, Deployment and Pending Work

Complete feature catalogue of VoiceSum with the technology behind each feature and where it lives in the codebase, followed by how to deploy it and what is still pending. Status as of 2026-09-14.

Paths are relative to the repository root. Backend API routes are served by `backend/main.py` on `127.0.0.1:8000`; frontend screens are routes in `frontend/src/App.tsx`.

1. Capture and input

No.	Feature	Description	Technology	Location
1	Live recording	Record a meeting from the microphone with a live waveform	Browser MediaRecorder, React	<code>frontend/src/pages/Record.tsx</code> , <code>components/VoiceRecorder.tsx</code> , <code>components/WaveformVisualizer.tsx</code> ; API <code>backend/routers/audio.py</code>
2	Browser tab audio capture	Record the audio of an online meeting running in another tab	Screen Capture API (<code>getDisplayMedia</code>)	<code>frontend/src/pages/TabAudio.tsx</code> , <code>hooks/useTabAudioRecorder.ts</code> , <code>services/recordingService.ts</code>
3	Audio file upload	Upload an existing recording for processing	FastAPI multipart upload	<code>frontend/src/pages/Upload.tsx</code> ; <code>backend/routers/audio.py</code>
4	Large file auto-chunking	Splits long recordings into chunks, transcribes each, and stitches timestamps back together	Python, <code>ffmpeg</code>	<code>backend/tasks/upload_chunk_pipeline.py</code> , <code>tasks/chunk_pipeline.py</code>
5	Audio trimming	Trim a recording before processing	React, Web Audio	<code>frontend/src/components/AudioTrimmer.tsx</code>
6	Video upload	Extracts the audio track from a video and runs the normal pipeline	<code>ffmpeg</code>	<code>frontend/src/pages/VideoUpload.tsx</code> ; <code>backend/routers/video_router.py</code> , <code>services/video_processing_service.py</code>
7	On-screen text from video	Samples frames, upscales and cleans them, and reads slide text into a timeline	OpenCV, super-resolution, RapidOCR (ONNX Runtime)	<code>backend/services/video_processing_service.py</code> , <code>services/ocr_engine.py</code> ; <code>components/VideoTranscriptViewer.tsx</code>
8	Agenda and context documents	Upload agenda, previous minutes or reference files to guide the minutes	<code>python-docx</code> , <code>python-pptx</code> , <code>PyMuPDF</code> , <code>pdfplumber</code>	<code>backend/routers/attachments_router.py</code> , <code>services/doc_extractor.py</code>

2. Speech processing

No.	Feature	Description	Technology	Location
9	Audio preprocessing	Loudness normalisation, adaptive voice activity detection, speech padding and low-volume recovery	librosa, soundfile, WebRTC VAD	backend/services/audio_preprocessing.py, utils/audio_utils.py
10	Transcription	Speech to text with a domain prompt and custom vocabulary	faster-whisper large-v3 (CTranslate2) via WhisperX	backend/services/transcription.py; model backend/runtime/models/speech_engine
11	Word-level timing	Aligns every word to the audio for precise timestamps	wav2vec2 forced alignment (WhisperX)	backend/services/transcription.py; model runtime/models/align_engine
12	Speaker diarization	Works out who spoke when, including overlapping speech	pyannote.audio 4 community-1	backend/services/diarization.py; model runtime/models/audio_context
13	Speaker identification	Matches each voice to enrolled people and corrects mislabelled segments	SpeechBrain ECAPA-TDNN embeddings, bipartite matching, refinement pass	backend/services/identification.py, services/embedding.py; model runtime/models/ecapa_tdn
14	Voice enrolment	Record voice samples to create a speaker profile, with specific error messages	ECAPA-TDNN, sample quality checks	frontend/src/pages/Setup.tsx, pages/AddVoice.tsx, components/VoiceTrainingModal.tsx; backend/routers/voice.py
15	Speaker management	Rename and merge speakers; a rename updates transcript, ROM and minutes together	FastAPI, SQLite	frontend/src/components/SpeakerTab.tsx; backend/routers/speaker_management_router.py, services/speaker_sync.py
16	Speaker re-identification	Re-runs diarization and identification on a finished recording after new enrolments	Same speech stack	backend/tasks/rereid_pipeline.py
17	Overlap detection	Detects segments where people talk over each other	pyannote	backend/main.py endpoint /api/detect-overlap
18	Transcript normalisation	Cleans filler, casing and shortcut expansions in the transcript	Rule-based Python	backend/services/transcript_normalizer.py
19	Processing pipeline	Orchestrates all stages with progress, cancellation and per-stage metrics	asyncio, FastAPI background tasks	backend/tasks/pipeline.py; components/GlobalJobTracker.tsx, components/ProcessingOverlay.tsx

3. Record of Meeting (ROM) and Minutes of Meeting (MoM)

No.	Feature	Description	Technology	Location
20	ROM Stage 1: point extraction	Extracts discussion points with speaker, entities, dates and figures from the transcript, with windowed re-runs	Local LLM via Ollama (Phi-4)	backend/services/rom_service.py, routers/rom_router.py (stage1 routes); frontend/src/pages/RomPage.tsx
21	ROM Stage 2: consolidation and editing	Merges duplicates; manual edit, split, merge, delete, find and replace, with edit history, undo and redo	Local LLM, BM25 plus semantic retrieval	backend/services/rom_service.py, routers/rom_router.py (stage2 routes); components/Stage2ContextPreviewModal.tsx
22	ROM Stage 3: agenda mapping	Maps points to agenda items, uses uploaded agenda and previous minutes	Local LLM, RAG	backend/routers/rom_router.py (stage3 routes); components/AgendaTimelineControls.tsx
23	Short and medium versions	Condensed ROM versions that keep provenance, metadata and a record of dropped points	Local LLM, provenance merge	backend/services/rom_service.py (generate_rom_version); route final/generate-version
24	Importance scoring	Scores each point on seven signals so condensing keeps what matters	Rule-based scoring	backend/services/point_importance.py
25	Action points and owners	Extracts actions with owners, due dates and how each owner was determined	Local LLM, evidence-based owner resolver	backend/services/rom_service.py; frontend/src/components/ActionItemsTable.tsx
26	Writing rules and rewrite	Learns house writing style from edits and rewrites the final ROM to it	Local LLM	backend/routers/rom_router.py (final/extract-writing-rules, final/rewrite)
27	Minutes of Meeting	Generates formal minutes with title, introduction, discussion, actions and conclusion	Local LLM, prompt templates	backend/routers/mom_router.py, routers/raw_mom_router.py, services/rag_pipeline.py; frontend/src/pages/MomPage.tsx, components/MomSection.tsx
28	Word export	Downloads each ROM stage, agenda transcript and minutes as Word documents	python-docx	backend/routers/rom_router.py (download/docx routes), routers/mom_router.py
29	PDF meeting report	Professional report with transcript, speakers, analytics and minutes	ReportLab	backend/routers/pdf_router.py; components/ExportPDFModal.tsx, components/PDFButton.tsx
30	Transcript review	Review and correct transcript text and speakers inline	React	frontend/src/components/TranscriptReviewPanel.tsx, TranscriptViewer.tsx, InlineEdit.tsx

4. Knowledge, search and AI assistant

No.	Feature	Description	Technology	Location
31	Global knowledge base	Organisation-wide documents used as background for every meeting	ChromaDB, mxbai-embed-large-v1	frontend/src/pages/GlobalContext.tsx; backend/routers/global_context_router.py, services/vector_store.py
32	Hybrid retrieval (RAG)	Finds relevant context by meaning, keywords and metadata, fused by rank	Semantic search, BM25+, reciprocal rank fusion	backend/services/rag_pipeline.py, services/bm25.py, services/text_chunker.py, services/text_embedding_service.py
33	AI chat on a meeting	Ask questions about a recording	Local LLM, RAG	frontend/src/components/AIChatPanel.tsx; backend/services/ai_provider.py
34	Collections	Group meetings into folders	FastAPI, SQLite	frontend/src/components/collections/; backend/routers/collections_router.py
35	AI chat across a collection	Ask questions spanning all meetings in a collection	Local LLM, RAG	components/collections/CollectionAIChat.tsx; backend/routers/collection_ai_router.py, services/collection_ai_service.py
36	Shortcut dictionary	Expands spoken abbreviations; CSV import and export	Python, FastAPI, SQLite	frontend/src/pages/Dictionary.tsx; backend/routers/dictionary_router.py, services/dictionary_service.py
37	Technical vocabulary	Domain terms that steer transcription, extracted from documents by rules or AI	Rule-based and LLM extraction, pandas	backend/routers/dictionary_router.py, services/vocab_extractor.py
38	Meeting history	List, open, search and delete past recordings	React Query, FastAPI	frontend/src/pages/History.tsx, pages/HistoryDetail.tsx; backend/routers/history.py

5. AI configuration and training

No.	Feature	Description	Technology	Location
39	Local language model	All generation runs on a local model server; picks the first installed model by priority; fails rather than hangs	Ollama on loopback, Phi-4 default, streaming with stall timeout	backend/services/ai_provider.py, services/llm.py; backend/config.py
40	Prompt templates	System-wide editable prompts for every AI step	SQLite, FastAPI	backend/routers/prompt_templates_router.py, routers/prompt_router.py, services/prompt_service.py, services/prompt_builder.py
41	Prompt optimisation (Stages 1 to 3)	Improves prompts from reviewer edits and feedback, with versioned variants	DSPy	frontend/src/pages/Training.tsx; backend/routers/training_routes.py, services/training/stage1_training_service.py to stage3_training_service.py; data backend/checkpoints/
42	Model fine-tuning	LoRA and 4-bit QLoRA fine-tuning jobs with datasets and evaluation	PEFT, bitsandbytes, transformers	backend/services/training/lora_training_service.py, qlora_training_service.py, training_service.py
43	User settings	Per-user thresholds, model priority, embedding model and processing options	FastAPI, SQLite, startup migration	frontend/src/pages/Settings.tsx, components/AdvancedOptions.tsx; backend/routers/settings_router.py

6. Security, administration and quality

No.	Feature	Description	Technology	Location
44	Login and sessions	Registration, login, token refresh and logout; session expiry prompt	JWT (python-jose), bcrypt, HttpOnly refresh cookie	backend/routers/auth.py; frontend/src/pages/Login.tsx, pages/Signup.tsx, components/SessionExpiredModal.tsx
45	Administrator role	First account is administrator; dashboard, logs, analytics and maintenance are admin-only	FastAPI dependencies, WebSocket cookie auth	backend/routers/auth.py (require_admin), routers/dashboard_router.py
46	Monitoring console	Live system status, logs, diagnostics export and maintenance actions	FastAPI, WebSocket	backend/routers/dashboard_router.py
47	Processing analytics	History of processing jobs, durations and outcomes for administrators	SQLite, FastAPI	backend/routers/analytics_router.py, services/analytics.py, services/run_metrics.py
48	Licence check	Blocks the application after a fixed expiry date	Python middleware	backend/license.py, backend/main.py; frontend/src/pages/LicenseExpired.tsx
49	Offline enforcement	Forces library telemetry and hub access off; tests fail if an external host appears	Environment controls, pytest	backend/offline_env.py, backend/tests/test_offline_egress.py
50	Per-install secret key	Generates a unique login-signing key on first run	Python secrets	backend/config.py; stored backend/runtime/secrets/
51	Golden dataset evaluation	Scores transcription, speaker, ROM retention and action accuracy against human references	jiwer, pyannote.metrics	backend/eval/ (runner.py, metrics/, golden/)
52	Automated tests and CI	728 backend tests; checks on every GitHub push	pytest, Ruff, Vitest, GitHub Actions	backend/tests/, .github/workflows/ci.yml

7. Platform and delivery

No.	Feature	Description	Technology	Location
53	Web frontend	Single-page interface with sketch-style design system	React 18, TypeScript, Vite, Tailwind, shadcn/Radix UI, React Query, Zustand	frontend/src/
54	Backend API	REST and WebSocket API with async database access	FastAPI, SQLAlchemy async, SQLite (WAL)	backend/main.py, backend/database.py, backend/routers/
55	Desktop application	Windows desktop shell serving the frontend over a private protocol	Electron 28	frontend-electron/main.js
56	One-command setup	Creates the environment, installs GPU or CPU PyTorch, fetches models and Phi-4, verifies	Python	setup.py, SETUP.md
57	Windows installer build	Compiles backend, launcher and desktop app into an installer	PyInstaller, electron-builder, Inno Setup	tools/, installer/setup.iss, BUILD.md
58	Offline updater	Patch and update tooling for installed copies	Python, PyInstaller	tools/updater.py, tools/create_patch.py, docs/update-guide.html

8. Deployment

Option	Use when	Steps	Status
A. Connected setup	A machine with internet during installation only, such as a demo GPU laptop	1. Install Python 3.12, Node.js 20, Git and Ollama. 2. Clone the repository. 3. Run <code>python setup.py</code> , which detects an NVIDIA GPU and installs the matching PyTorch, downloads the speech and embedding models, and pulls Phi-4. 4. Run <code>python setup.py --check</code> . 5. Start the backend with the virtual environment's Python and the frontend with <code>npm run dev</code> , or build the frontend for the desktop app. After setup the machine can be disconnected.	Working and used on this laptop
B. Air-gapped machine	The target never has internet	On a connected build machine of the same Windows and Python version: 1. Run option A. 2. Download all Python packages to a folder with <code>pip download</code> . 3. Copy the repository, that package folder, <code>backend/runtime/models</code> , <code>backend/runtime/embeddings</code> , the Ollama installer and the Ollama models folder to removable media. 4. On the target, install Python, Node.js and Ollama from the media, install packages with <code>pip install --no-index</code> , copy the models into place and run <code>python setup.py --check</code> .	Manual; a verified offline bundle script is pending (W4.5)
C. Packaged installer	Delivery to end users who should not see source code	Follow <code>BUILD.md</code> : build the frontend, package Electron, compile the backend and launcher with PyInstaller, and build the installer with Inno Setup. Place models and Ollama separately.	Build guide is out of date: it still describes a Qwen model folder, encrypted model files and Whisper medium. Must be updated and rebuilt before use

Hardware guidance: an NVIDIA GPU is required for practical use of Phi-4. A sizing estimate, not yet measured, is 12 GB of video memory or more. On this 16 GB CPU-only laptop the speech pipeline ran a 15-second recording correctly offline, but a single Phi-4 request took several minutes.

9. Pending

Priority	Item	Why it matters	Owner
Critical	Licence expiry is hard-coded to 30 September 2026	The application stops working after that date, 16 days from now. Change <code>LICENSE_EXPIRY_DATE</code> in <code>backend/license.py</code> for the release	Your decision on the new date
Critical	Real recordings for the golden dataset	No accuracy number can be claimed without them	Your team supplies recordings and checks references
Critical	Run the evaluation on a GPU machine	Phi-4 is impractical on this laptop	Run on the demo machine
High	Update and rebuild the Windows installer (deployment option C)	Build guide and tools still reference Qwen and an older model layout	Development
High	Offline installation bundle (W4.5)	Air-gapped installation is currently manual	Development
High	Replace RapidOCR (Baidu PP-OCR models) in video text reading	Model sourcing constraint	Development; needs a permitted OCR model
High	Review image upscaling models for video OCR	Optional Real-ESRGAN path comes from Tencent ARC	Development
High	Remove the in-process Qwen fallback and <code>tools/download_qwen.py</code>	Unused by default, but still present	Development
High	Load models once per run, crash recovery and stage checkpoints (W2.1 to W2.3)	Largest speed and reliability gains on long meetings	Development
High	Audit log of logins and changes, and keeping names out of logs (W6.2, W6.5)	Requested traceability	Development
Medium	Weekly leadership health report and admin console (W6.6 to W6.8)	Requested reporting	Development
Medium	Speaker split clusters, short replies, reassigned actions (W1.5, W1.6, W1.8)	Remaining accuracy issues; need real recordings to validate	Development
Medium	Show dropped points in the interface (W1.3)	Reviewers cannot yet see what condensing removed	Development
Medium	Introduction and conclusion shorter than five sentences are kept as is	Padding would add sentences nobody said	Your decision
Medium	Database migrations tool (W3.1)	Safe upgrades of installed data	Development
Low	Remaining workstreams	See <code>docs/WORKSTREAMS.md</code>	Development